7-rotaxane test (rotx7)

A larger proof-of-concept test than the rot1/rot2 mini tests in DEV.md: does training the ConSolv-style implicit-solvent MACE on GFN2/ALPB data improve binding energies across solvents, and which way of handling the vacuum/solvent split works best? Started 2026-11-05.

Data (data/rotx7/)

Systems: rot1, rot2htpuma, rot3, dethread1, dethread2, dethread3 (dethread3cf), dethread4 (104–144 atoms; wheel 24-crown-8 or dibenzo-24-crown-8). Structures (threaded starting geometries, GFN2 scan minima) in inputs/current_structures/ (gitignored). The rod is the bonded component holding atom 1, checked against each system's SMILES.

- **Sampling.** For each complex: GFN2 gas relax, then 1200 × 0.5 fs Langevin MD at 300 K, sampled every 30 steps → 42 frames (relaxed minimum + 41 MD).
- **Fragments.** Every complex frame is cut into rod and wheel at their geometries in that frame. All three pieces are labelled and used in training, matching how the binding energy is evaluated (rod and wheel at their in-complex geometries). No separately relaxed fragments.
- **Labels.** GFN2 gas phase, plus GFN2/ALPB in hexane, chloroform, acetone, DMF, water and DMSO (ConSolv solvent indices 25, 11, 7, 53, 1, 104), on identical geometries.
- **Split.** dethread4 is held out entirely as validation (its complex, rod and wheel frames). Train: 756 frames per label set (6 systems × 42 × 3 pieces); valid: 126.

Generated on the laptop serially in ~1 h (scripts/run_rotx7.sh). Training on Modal (scripts/modal_train.py, L4 with A10G fallback).

Models trained

All use the same data. They differ in what the network learns and where it starts.

run	network	starts from	learns	trainable	energy in solvent at use	epochs, lr
rotx7_vac	vacuum MACE	published vacuum checkpoint	gas-phase E, F	everything	(building block for (a), (b))	50, 2.5e-4
rotx7_solv	solvent-conditional MACE (MACE_CONSOL)	published ConSolv solvent checkpoint	total E_ALPB, F in each solvent	everything (768k params)	this network alone	20, 2.5e-4
(a) rotx7_delta	second MACE_CONSOL, alongside rotx7_vac	random init	solvent effect only: E_ALPB − E_gas and ΔF, same geometries	everything in the delta network; rotx7_vac untouched	E_vac(rotx7_vac) + E_delta	40, 1e-3
(b) rotx7_embed	one MACE_CONSOL	rotx7_vac weights in all shared layers; published solvent embedding block	total E_ALPB, F in each solvent	only the solvent embedding block (~133k params)	this network alone	20, 2.5e-4

- **rotx7_solv** is the baseline: nothing held fixed.
- **(a)** and **(b)** are two ways of holding the vacuum physics fixed at rotx7_vac. (a) adds a separate solvent correction on top. (b) lets the solvent modulate the per-atom embedding inside one network.
- **The published ConSolv checkpoint** is structurally closest to (b): one network initialised from the vacuum weights. Nothing was frozen, but the shared weights moved only ~0.2–2% while the solvent embedding block moved 7–65%. Its solvent part was fitted to experimental solvation free energies, not to energies and forces.

Common settings: batch 4, γ_U = 1, γ_F = 0.01, per-element E0 refit on the starting model's residuals (train.py).

How one model covers several solvents

The mechanism (same in every solvent-conditional network)

rotx7_solv, (a) rotx7_delta, (b) rotx7_embed and the published ConSolv model all use the same architecture (MACE_CONSOL, block_name = "descriptor") and receive the solvent the same way. rotx7_vac has no solvent input and is gas phase only.

- Each frame carries an integer solvent index, solv_species (ConSolv descriptor row + 1, from scripts/solvent_index.py: hexane 25, chloroform 11, acetone 7, DMF 53, water 1, DMSO 104; 0 is the all-zero vacuum row).

- The network looks up that solvent's 15 normalised RDKit descriptors. This is a fixed table, not a learned embedding, so no model has per-solvent parameters.
- The descriptors enter in two places. **(i) Node embedding:** each atom's element embedding attends to the projected descriptor vector (cross-attention block, ~133k params). The result is added to the atom embedding. **(ii) Interaction layers:** in both message-passing layers, two small linear layers (1.5k params per layer) map the descriptors to a factor that rescales the radial basis weighting the messages (solv_tp = true).
- **How one set of weights gives each solvent its own behaviour.** Every weight is used for every solvent, and no weight belongs to a particular one. The descriptor vector is combined with the atom features at (i) and multiplies into the message weights at (ii). Feeding in water's descriptors therefore draws water's behaviour out of the same weights that give hexane's behaviour from hexane's descriptors. The model is one function E(R, descriptors), with the solvent as an argument, the way temperature is an argument in a fit E(R, T). Training adjusts the weights until, for each training solvent, these combinations reproduce that solvent's energies and forces.
- Consequences: a solvent between the training solvents in descriptor space gets an interpolated response, and changing the shared weights for some solvents also changes the predictions for all the others (see "Catastrophic forgetting").

The published model trained these paths top-down, on experimental solvation free energies of small solutes in 66 solvents (DEV.md). It was not trained on energies or forces in each solvent.

1. How the fine-tuned models train on several solvents at once

The data layout is the same for all three. train.py --model solvent --sets hexane=25 chloroform=11 ... reads one labelled file per solvent (data/gfn2_<solvent>_{train,valid}.xyz), concatenates them and stamps every frame with its solvent's index. Each (geometry, solvent) pair is a separate training example, so one geometry R becomes six rows:

input	target
R, index 25 (hexane)	GFN2/ALPB E, F in hexane
R, index 11 (chloroform)	E, F in chloroform
R, index 7 (acetone)	E, F in acetone
R, index 53 (DMF)	E, F in DMF
R, index 1 (water)	E, F in water
R, index 104 (DMSO)	E, F in DMSO

That gives 756 geometries × 6 = 4,536 rows. They are shuffled together, so a batch of 4 can hold, for example, geometry 12 in water, geometry 300 in hexane, geometry 12 in DMSO and geometry 55 in acetone. Each row is scored against its own label, and all rows update the same weights. For (a) the targets are the solvent values minus the gas values of the same R. There is one loss ($\gamma_U \cdot \mathrm{MSE}(E) + \gamma_F \cdot \mathrm{MSE}(F)$) and one set of weights. On identical geometries, the solvent effect is the only thing that differs between copies, and the index is the only input that differs. The per-element E0 shift is fitted once for all solvents, so it cannot absorb solvent-to-solvent differences.

The three fine-tunes differ in their targets, their starting weights and which parts may move:

run	starts from	target per solvent copy	trained	solvent paths (i) attention, (ii) radial
rotx7_solv	published ConSolv weights (66 solvents, free energies)	total E_ALPB, F	all 768k params	both retrained; shared layers move too
(a) rotx7_delta	random init	E_ALPB – E_gas, F_ALPB – F_gas (--delta-from gas)	all params of the delta network	both learned from scratch, from these 6 solvents only
(b) rotx7_embed	rotx7_vac in shared layers; published weights elsewhere	total E_ALPB, F	(i) only (--train-only non_linear_solvent_embedding)	(i) retrained; **(ii) frozen at published values**

- **rotx7_solv** begins with a solvent response learned from free energies and reshapes it to the ALPB energies and forces. Every weight is shared, so the outputs for the 60 solvents it was not fine-tuned on also change. That change is untested.
- **(a)** has no prior. Its whole notion of "solvent" comes from 6 points in descriptor space. It learns only the solvent correction, because the gas-phase part is subtracted from each target. Index 0 (vacuum) was never in its training data.
- **(b)** The (ii) radial-modulation layers do not exist in the vacuum model, so --init-vacuum left them at the published values. They also do not match the --train-only substring, so they stayed frozen. Only path (i) adapted. This is a possible contributor to (b)'s poor result (Conclusion 3), not tested separately.

2. Choosing a solvent at inference

In every case you pass the solvent's index with the coordinates and the weights stay the same. binding_energy.py gives every atom solv_species = <index> (jnp.full(n, solv)) and relaxes and evaluates once per listed solvent:

model	call	energy evaluated
rotx7_solv	--params runs/rotx7_solv --solvents water=1 dmso=104 ...	network(geometry, index)
(a)	--params runs/rotx7_delta --vacuum-params runs/rotx7_vac --solvents ...	E_vac(geometry) + E_delta(geometry, index); only the delta term sees the solvent
(b)	--params runs/rotx7_embed --solvents ...	network(geometry, index)
gas phase	--params runs/rotx7_vac --solvents gas=0	vacuum network; the index is ignored

- One trained model therefore gives a separate potential energy surface for each index. Changing solvent means changing one integer.
- Any of the 147 descriptor rows can be passed. For the 6 training solvents the result is fitted. For any other solvent it is an interpolation through the descriptors: from the published 66-solvent prior plus the fine-tune for rotx7_solv, and from 6 solvents alone for (a). Neither has been tested (see "Catastrophic forgetting" below).
- For (a), use rotx7_vac alone for the gas phase, not the delta network with index 0.
- Indices for new solvents come from their SMILES: scripts/solvent_index.py toluene=Cc1ccccc1.

Results

Held-out validation (dethread4), energies and forces

run	E RMSE (kcal/mol/frame)	F RMSE (kcal/mol/Å)
published vacuum, after E0 refit (gas)	4.89	4.87
rotx7_vac (gas)	1.80	2.75
published solvent, after E0 refit	6.78	5.07
rotx7_solv	1.95	2.65
(a) rotx7_delta, on the delta E_ALPB – E_gas only (not comparable)	0.51	0.54
(b) rotx7_embed	3.04	3.15

Held-out validation, solvent differences

E(solvent) – E(hexane) on identical geometries, RMSE vs GFN2/ALPB (kcal/mol). The vacuum energy cancels in this difference, so all four columns are directly comparable.

solvent	GFN2/ALPB mean	published	rotx7_solv	(a)	(b)
chloroform	–3.02	3.69	1.94	0.39	1.68
acetone	+1.34	7.58	0.81	0.29	1.66
DMF	+1.50	6.09	1.35	0.31	2.09
water	+17.12	16.52	0.94	0.67	5.26
DMSO	+1.46	9.62	1.53	0.28	2.24

Binding energies, dethread4 (held out)

BE = E(complex) – E(rod) – E(wheel), kcal/mol. The complex is relaxed with each model; rod and wheel are single points at their geometries in the relaxed complex. Reference: GFN2/ALPB single points on each model's relaxed geometries.

solvent	published	GFN2/ALPB (pub. geom)	rotx7_solv	GFN2/ALPB	(a)	GFN2/ALPB	(b)	GFN2/ALPB
hexane	–33.49	–21.21	–28.23	–21.50	–28.41	–20.93	–33.90	–22.61
chloroform	–30.39	–17.69	–23.82	–17.99	–24.46	–16.98	–31.03	–19.37
acetone	–27.99	–15.45	–21.67	–15.27	–22.46	–14.37	–28.50	–16.13
DMF	–27.28	–15.21	–20.65	–14.88	–22.17	–13.88	–28.10	–16.00
water	–30.04	–26.12	–31.36	–26.07	–32.24	–25.59	–28.22	–29.34
DMSO	–28.75	–15.08	–21.36	–15.07	–22.35	–13.93	–28.09	–16.18
MAE vs own GFN2/ALPB	**11.20**		**6.05**		**7.74**		**10.08**	
MAE of BE(solv) – BE(hexane)	2.12		0.82		**0.64**		3.06	
spread (max – min)	6.2	11.0	10.7	11.2	10.1	11.7	5.8	13.3

- **Ordering.** GFN2/ALPB: water > hexane > chloroform > acetone > DMF ≈ DMSO. `rotx7_solv` reproduces it exactly. Published: hexane > chloroform > water > DMSO > acetone > DMF, with water in the wrong place.
- **Offset.** `rotx7_solv` over-binds by a nearly uniform 5.3–6.7 kcal/mol in every solvent; the published model over-binds by 12–14 kcal/mol (3.9 in water). The constant part looks like the vacuum interaction (as in the rot1/rot2 mini test), not the solvent response.
- The reference moves < 0.4 kcal/mol between the published and `rotx7_solv` geometries; on the (b) geometries it moves more (water −29.3, hexane −22.6).
- **(a) `rotx7_delta` + `rotx7_vac`** has the best solvent response: BE(solvent) − BE(hexane) within 0.64 kcal/mol on average, correct ordering, spread 10.1 vs 11.7. But it over-binds by a near-constant 6.7–8.4 kcal/mol (MAE 7.74), more than `rotx7_solv` (6.05). The delta model is accurate (validation ΔE RMSE 0.3–0.7), so the offset comes from the vacuum model. Measured: gas-phase BE of dethread4 with `rotx7_vac` is −37.42 vs GFN2 −31.02 kcal/mol, i.e. 6.4 kcal/mol of over-binding already in vacuum, most of (a)'s 6.7–8.4. The full fine-tune hides part of that offset in its solvent weights, which is why its total MAE is lower.
- **(b) `rotx7_embed`** behaves like the published model: over-binds by 11–12 kcal/mol in the organic solvents, water too weak (+1.1), order hexane > chloroform > acetone > water > DMF ≈ DMSO, spread 5.8 vs 13.3. Training only the solvent embedding on fixed `rotx7_vac` weights does not carry the solvent response.

Notes for production training

- **Labels: DFT, not GFN2.** Production runs will be labelled with DFT, so the reference level is the published vacuum model's: **ωB97M-D3(BJ)/def2-TZVPPD** (SPICE, as used by MACE-OFF; upstream vacuum config `source = "spice_mace_off"`). Use exactly this level for compatibility. Fine-tuning will then refine the published model instead of moving it to a different level of theory. That should ease the E0 offset and the pressure on the shared layers seen here with GFN2.
- **Solvent labels: DFT + SMD.** No experimental solvation free energies exist for the rotaxanes, so ConSolv's top-down fit is not available. Label each frame in gas and in each solvent with ωB97M-D3(BJ)/def2-TZVPPD + SMD on identical geometries (GPU4PySCF has PCM/SMD). SMD over plain PCM because it has the non-electrostatic (cavity/dispersion) terms, which drive the water trend seen with ALPB. Same layout as rotx7, so both the full fine-tune and the delta scheme (a) stay possible. Cost per frame: 1 + N_solvents SCFs (seed solvent SCFs with the gas density).
- **Codes.** GPU: GPU4PySCF (PCM/SMD). CPU (lab machines): ORCA (`wB97M-D3BJ def2-TZVPPD`, SMD via the CPCM module). Keep one code per training set: RI (ORCA's RIJCOSX), grids and SMD cavity construction differ, and mixed labels add code-to-code noise to the small solvation differences. If both are used, first label ~20 shared frames with both and compare E, the gas→solvent ΔE and F. (SPICE itself was computed with Psi4.)
- **Catastrophic forgetting (not tested in this POC).** The full fine-tune (`rotx7_solv`) trains every weight on 6 solvents and 6 related rotaxanes. It can degrade the published model's other ~60 solvents and its general chemistry. For production, train on the rotaxane frames plus a small subset of the original training data (SPICE frames; ConSolv solvation data) to keep it. A cheap check, if wanted: relabel held-out frames in unseen ALPB/implicit solvents (DCM, toluene, methanol, acetonitrile, THF) and compare E(solvent) − E(hexane) with the published model.

Conclusions

1. **Training on GFN2/ALPB data works and generalises to a held-out rotaxane.** On dethread4, which no model saw, the full fine-tune (`rotx7_solv`) halves the published model's BE error (11.2 → 6.1 kcal/mol) and recovers the GFN2/ALPB solvent ordering, including water as the strongest binder, which the published model gets wrong.
2. **The vacuum + delta split (a) gives the best solvent response.** Solvent-to-solvent BE changes are within 0.64 kcal/mol of GFN2/ALPB and the held-out ΔE(solvent − hexane) RMSE is 0.3–0.7 kcal/mol, 2–5× better than the full fine-tune. Its larger total BE error is the vacuum model's: `rotx7_vac` already over-binds dethread4 by 6.4 kcal/mol in the gas phase.
3. **Training only the solvent embedding (b) does not work.** With the shared layers fixed at the vacuum weights, the model stays close to the published one (MAE 10.1, wrong water placement). The solvent response needs more than the embedding block, at least when the reference level changes as here (ωB97M → GFN2). Caveat: the run also froze the second solvent path (the radial modulation in the interaction layers) at the published values, so this was not a clean test of "embedding only".
4. **The remaining error is the vacuum interaction energy**, not the solvent model. The next improvement is a better vacuum model for host–guest interaction (more and more varied complex frames; in production, DFT labels at the published level, ωB97M-D3(BJ)/def2-TZVPPD).
5. **Recommendation for production:** the delta scheme (a), i.e. a vacuum model plus a solvent model trained on E_solv − E_gas on identical geometries, with DFT + SMD labels. It isolates the solvent response, keeps vacuum errors diagnosable, and the delta network trained in under an hour on one L4.

Caveats: one held-out system, GFN2 rather than DFT reference, binding energies (not free energies), and no test of forgetting on unseen solvents or chemistry.